Solution Description

Problem Statement (PS-4)

AI-enhanced Application for Automated Data Preparation, Estimation and Report Writing Ministry of Statistics & Programme Implementation (MoSPI), Government of India Track: Data Processing and Analysis

---

Problem Being Solved

Official statistical agencies like MoSPI work with NSS (National Statistical Survey) microdata that arrives as a set of cryptically-named raw files:

- A **CSV** of microdata where every column is a short code (e.g. `b11c4`) with no readable meaning
- An **Excel Data Layout** mapping those codes to field names and block numbers
- A **Schedule PDF** containing the question text and value labels (e.g. `1 = Government Hospital, 2 = PHC`)

Before any analysis can happen, analysts must manually cross-reference all three files. This is laborious, error-prone, and non-reproducible. On top of that, producing a final statistical report — with weighted estimates, margins of error, charts, and narrative — requires significant manual effort each time.

Our Solution

A fully automated, AI-augmented Flask web application that takes the three raw input files and outputs a publication-ready HTML/CSV report — without the analyst needing to look at the Data Layout or Schedule PDF at all.

How It Addresses Each Required Feature

1. Data Input & Configuration

Requirement	**Implementation**
CSV/Excel upload	Step-by-step file upload UI with drag-and-drop, progress indicators, and instant validation feedback
Schema mapping via UI or JSON config	Automatic schema mapping from the Data Layout Excel; dimensions, measures, and weight columns are configurable through the table-builder UI

2. Cleaning Modules

Requirement	Implementation
Missing-value imputation (mean, median, KNN)	Preprocessing step lets the analyst choose the imputation strategy per column type; applied before estimation
Outlier detection (IQR, Z-score, winsorization)	Configurable outlier detection with flagging and replacement options; `outlier_detector` module handles all three methods
Rule-based validation (consistency, skip-patterns)	Skip-pattern and range-based rule checks are applied during the consolidation step using the schedule metadata

3. Weight Application

Requirement	Implementation
Apply design weights	The weight column (survey multiplier) is auto-detected from the dataset or specified by the user; used for all estimation
Compute weighted summaries and margins of error	Linearisation (sandwich) variance estimator computes a true 95% MOE for every Weighted Mean measure: $MOE = 1.96 \times \mathrm{sqrt}(\Sigma(w_i^2 \times (x_i - \hat{\mu})^2) / (\Sigma w_i)^2)$

4. Report Generation

Requirement	Implementation
Auto-generate reports using templates	One click generates a fully formatted HTML report with cover, methodology, data tables, visualisations, and AI-written narrative
Workflow logs, diagnostics, and visualisations	Each report includes Plotly interactive charts (auto-selected bar, donut, stacked-bar, or line based on data type), table diagnostics, and a Trend Summary section
AI-generated narrative	Groq (`llama-3.3-70b-versatile`) writes Analysis, Query Answer, Key Insights, and Recommendations; Google Gemini writes the Trend Summary. All AI outputs are cached to disk to avoid redundant API calls

5. User Guidance

Requirement	**Implementation**
Tooltips, inline explanations, error-checking alerts	Each step shows inline guidance, field-level tooltips, and error banners (e.g. missing weight column, empty filter results)

Architecture Overview

```
User Uploads (CSV + Excel + PDF)
        │
        ▼
┌──────────────────────────────┐
│  Step 1-3: Ingestion         │  pandas (CSV), openpyxl (Excel), pdfplumber
(PDF)
│  - Dataset metadata          │
│  - Layout mapping            │
│  - Value label codebook      │
└──────────────────────────────┘
             │
             ▼
┌──────────────────────────────┐
│  Step 4: Consolidation       │  Resolves variable names + value codes
│  consolidated_resolved_      │  → single analysis-ready CSV
│  dataset.csv                 │
└──────────────────────────────┘
             │
             ▼
┌──────────────────────────────┐
│  Step 5: Cleaning            │  Imputation, outlier detection, rule checks
│  preprocessed_dataset.csv   │
└──────────────────────────────┘
             │
             ▼
┌──────────────────────────────┐
│  Step 6: Estimation          │  Weighted means, frequencies, ratios
│  + 95% MOE                   │  Linearisation variance estimator
│  + Auto chart selection      │  Plotly interactive charts
└──────────────────────────────┘
             │
             ▼
┌──────────────────────────────┐
│  AI Report Writer            │  Groq (primary) → Gemini (fallback)
│  - Analysis                  │  Disk-cached by SHA-256 prompt hash
│  - Insights                  │  (saves tokens on re-runs with same data)
│  - Recommendations           │
│  - Trend Summary             │
└──────────────────────────────┘
             │
             ▼
   HTML Report (editable, downloadable)
```

Key Technical Differentiators

- **No manual cross-referencing** — The three-file integration (CSV + Excel + PDF) is fully automated using regex, pandas, and pdfplumber.
- **Correct statistical estimation** — Uses the proper linearisation variance estimator for survey-weighted MOE, not a simple standard deviation.
- **AI with caching** — Groq/Gemini responses are cached by prompt hash to prevent repeated API calls for the same data, reducing cost and latency.
- **Editable reports** — The generated HTML report is `contenteditable`, letting analysts refine the AI text before publishing.
- **Bonus: Admin dashboard** — Role-based access with an admin dashboard tracking all users and report activity (audit trail).

Relevance to MoSPI's Objectives

This solution directly supports MoSPI's goal of improving data quality and efficiency through automation and AI. It:

- Reduces the time from raw survey file delivery to publishable report from days to minutes
- Enforces methodological consistency (same weighting formula, same codebook, same template every time)
- Reduces human error in variable-name lookup and value-label decoding
- Produces reproducible outputs (same inputs → same tables → cached AI text → identical report)